

Project 1 Report: Video Special Effects

Computer Vision

May 27, 2026

Project Description

This project implements a real-time video filtering program. The program captures frames from a camera, processes each frame using custom image manipulation functions in `filter.cpp`, and displays the selected result interactively. The required filters include custom greyscale, sepia tone, 5x5 blur, Sobel X and Y, gradient magnitude, blur and quantization, face detection, and depth-based effects using Depth Anything V2 through ONNX Runtime. I also implemented additional special effects, including negative color, embossing, depth fog, and cartoonization. The main goal was to practice direct pixel access, convolution-style neighborhood operations, separable filtering, live video interaction, and combining classical computer vision with a neural depth model.

Required Results

Required Image 2: Custom Greyscale



(a) Default OpenCV greyscale



(b) Custom greyscale

Figure 1: The custom greyscale filter uses the expression $255 - \text{red} + (\text{green} - \text{blue}) / 4$, then copies the result into all three BGR channels. This makes the output a true greyscale image because all channels are identical, but it looks different from OpenCV greyscale because it emphasizes inverse red intensity and a small green-blue contrast term instead of using a standard luminance-weighted average.

Required Image 3: Sepia Tone



Figure 2: Sepia tone with vignette. The filter stores the original B, G, and R values before computing the new values, so the red result is never reused while calculating green or blue. Each output channel is clamped to 255, and a mild vignette darkens pixels farther from the image center.

Required Image 4: 5x5 Blur and Timing



(a) Original frame



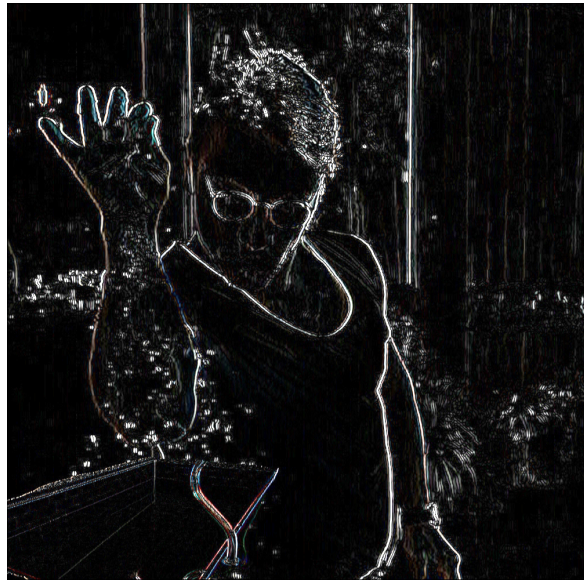
(b) 5x5 blur

Figure 3: The blur filter processes each color channel separately using the integer Gaussian approximation. The naive version uses a full 5x5 nested loop and `at`. The faster version uses separable 1x5 horizontal and vertical passes with pointer access. On `pathfinder.png` with 50 iterations, `blur5x5_1` averaged 694.959 ms and `blur5x5_2` averaged 113.624 ms, a 6.12x speedup.

Required Image 5: Sobel and Gradient Magnitude



(a) Original



(b) Sobel X



(c) Sobel Y



(d) Gradient magnitude

Figure 4: The Sobel X and Sobel Y filters are implemented as separable 1x3 filters and store signed short color outputs. Sobel X emphasizes vertical edges, Sobel Y emphasizes horizontal edges, and the magnitude image computes $\sqrt{s_x^2 + s_y^2}$ for each color channel before converting to displayable unsigned char values.

Required Image 6: Blur and Quantization



(a) Original



(b) Blur and quantize

Figure 5: The blur-quantize effect first applies the faster 5x5 blur, then reduces each channel to a fixed number of buckets. With 10 levels, the output contains a smaller set of possible colors, giving a simplified poster-like appearance.

Required Image 7: Face Detection



Figure 6: Face detection uses the provided Haar cascade. The video program converts the frame to grayscale for detection, then draws rectangles around detected faces when face display is enabled.

Required Images 8–9: Depth Anything V2



(a) Depth map



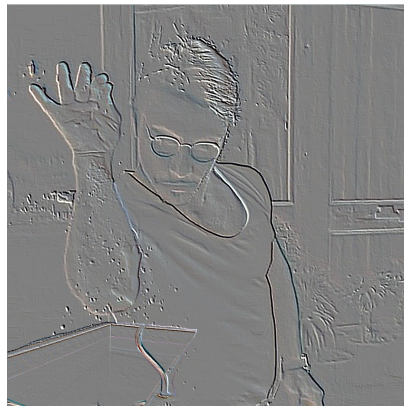
(b) Depth-based fog

Figure 7: Depth Anything V2 estimates a relative depth value for every pixel from a single RGB frame. The depth image is colorized for display. The creative depth filter adds fog as an exponential function of the normalized depth value, mixing distant/deeper pixels with a pale fog color more strongly than nearer pixels.

Required Images 10–12: Additional Effects



(a) Negative



(b) Emboss



(c) Cartoon

Figure 8: The negative effect is a single-step pixel-wise modification that replaces each channel with $255 - \text{value}$. Emboss uses an area computation by combining Sobel X and Sobel Y values with a middle gray offset to create a raised-relief look. Created the cartoon filter based on the paper. .

Reflection

This assignment helped me understand how to manipulate pixel values to achieve the filters. I also looked at GitHub for projects like Depth Anything v2 and v3, ONNX, and face recognition, and I'm

interested in learning more about how these work with the underlying neural networks, since this time I used them like a black box. The 5×5 blur comparison made the performance impact of memory access patterns very clear: the separable pointer-based version was much faster than the naive nested-loop version. I also learned why signed intermediate images are important for edge filters, since Sobel responses can be negative.

Acknowledgements

I consulted the OpenCV documentation for pixel access, display, image conversion, and `convertScaleAbs`. I used the provided course files for face detection and the Depth Anything V2 wrapper, along with the SFace model from the OpenCV Model Zoo (https://github.com/opencv/opencv_zoo/tree/main/models/face_recognition_sface). For the cartoonization extension, I referred to Winnemöller, Olsen, and Gooch's *Real-Time Video Abstraction* (ACM Transactions on Graphics, 2006), I used LLM to help me understand this paper and to streamline the application of this filter. I also relied on Homebrew-installed OpenCV, ONNX Runtime while building and testing the project.

1 Video Demo

Here is a video running the video filter demo <https://www.loom.com/share/213a8b6afbbe4afe8aa588d56c599a8d>